

tvscientificapp

April 27, 2025

```
[17]: # Note: This document is read-only. Copy the questions into a
# Jupyter notebook and print a pdf for submission.
# Please keep these questions confidential.

## General Guidance
#
# - Strong candidates will usually be able to finish these questions
#   in 15 minutes or less, without using any AI code assistance tools.
# - Assume that inputs to the functions you write are sanitized.
# - Concise, expressive, functional answers are encouraged.
# - You do not need to write documentation, use type annotations, etc.
```

```
[18]: # Q1) Define a function that returns the frequencies of the last digits
#      of a list of nonnegative integers.
#
#      Use only the Python standard libraries.
#
#      Given the list [49, 10, 20, 5, 30, 785]:
#      9 is the last digit once (in 49),
#      0 is the last digit three times (in 10, 20, and 30),
#      5 is the last digit two times (in 5 and 785)
# last_digit_counts([49, 10, 20, 5, 30, 785])
# = {9:1, 0:3, 5:2} # or something equivalent

from collections import defaultdict
def last_digit_counts(nums):
    counts = defaultdict(int)
    for num in nums:
        counts[num % 10] += 1
    return dict(counts)
```

```
[19]: print(last_digit_counts([49, 10, 20, 5, 30, 785]))
```

```
{9: 1, 0: 3, 5: 2}
```

```
[20]: # Q2) Given a list of 2-D coordinates, write a function that returns True
#      if the points lie on a straight line and False otherwise.
#
```

```

#     Use only the Python standard libraries.
#
# collinear([[1,1], [2,2], [4,4], [-10, -10]]) = True
# collinear([[1,0], [2,0], [3,1]]) = False

def collinear(points):
    if len(points) < 2:
        return True

    xchange = 0
    ychange = 0
    differentpoints = False
    for i in range(1, len(points)):
        if points[i] != points[0] and not differentpoints:
            ychange = points[i][1] - points[0][1]
            xchange = points[i][0] - points[0][0]
            differentpoints = True
        else:
            if xchange != 0:
                if (points[i][0] - points[i-1][0]) != 0:
                    if (points[i][1] - points[i-1][1]) / (points[i][0] -
↪points[i-1][0]) != ychange/xchange:
                        return False
                else:
                    return False
            else:
                if points[i][0] - points[i-1][0] != 0:
                    return False

    return True

```

```

[21]: print(collinear([[1,1], [2,2], [4,4], [-10, -10]]))
      print(collinear([[1,0], [2,0], [3,1]]))
      print(collinear([[0,1], [1,1]]))
      print(collinear([[0,1], [0,2]]))
      print(collinear([[0,0], [0,0], [1,1], [3,2]]))
      print(collinear([[0,0], [0,0], [1,1], [3,3]]))

```

```

True
False
True
True
False
True

```

```

[22]: # Q3) If you flip a fair coin 100 times (independently),
      #     what is the chance that more than 60 flips come up heads?
      #     Do not try to compute this value exactly; instead, use

```

```
# the simplest "good" approximation you can come up with.
# You should not write any code for this problem.

# Expected value of 1 coin flip: 0.5 Heads
# Variance of 1 coin flip: 0.25
# => EV of 100 coin flips: 50, Variance of 100 coin flips: 25
# Where S is the sum of all 100 coin flips (T = 0, H = 1),
# CLT says  $(S - E(S))/\sqrt{\text{Var}(S)} \rightarrow \text{Normal}(0,1)$ 
# Hence, 60 H is ~ 2 standard normal deviations above average, so the
  ↪ probability should be about 2.5%
```

```
[ ]: # Q4) What tools do you use to help you code productively?
# For example, what editor/IDE do you prefer?

# For larger files: VSCode; for scripting: Command line

# Testing framework? Linter? Command line utilities?

# Too inexperienced with SWE to have a consistent testing framework
# Linter: Fish for bash, Copilot for VSCode (although I'd like to transition to
  ↪ something free and local over Copilot eventually)
# Command line utilities: I'm probably too much of a noob to give the "right"
  ↪ answer to this;
# I use btop, mpv, and git from command line I guess but mpv's only utility is
  ↪ giving me dopamine from music

# Other tools you like or recommend to friends?
# Is there something you've discovered recently but haven't
# had time to learn yet?

# I want to better learn convex optimization, especially since I'm currently
  ↪ teaching it
# I'm starting to learn C++ but very slowly; I'd like to develop a trading
  ↪ system using https://github.com/lballabio/QuantLib
# once I'm more proficient with C++; only discovered QuantLib earlier today.

# Of note that I am a "fresh meat" math PhD so my strengths lie in my
  ↪ quantitative precision
# more than my coding background; if you need a SWE more than a researcher then
  ↪ I may not be your best choice

# Time taken: 30 minutes; 15 answering Q1-3, 10 answering Q4, 5 figuring out
  ↪ how to turn this file into a pdf
```